

ASSIGNMENT 4.2.

a.

```
[1]: # Nguyễn Hoàng Tùng

import pandas as pd
import numpy as np
import random
import os

# 1. Create folder if not exists
os.makedirs("C:/DATA", exist_ok=True)

# 2. Define number of samples and features
n_samples = 10000

# 3. Generate synthetic data
np.random.seed(42) # for reproducibility

# Jobs (choose from a fixed list)
jobs = ["Engineer", "Doctor", "Teacher", "Farmer", "Driver", "Nurse",
        "IT Specialist", "Lawyer", "Artist", "Student"]

# Areas (urban, rural, suburban)
areas = ["Urban", "Rural", "Suburban"]

# Diet type
diets = ["Balanced", "High-Protein", "Vegetarian", "Keto", "Low-Carb", "Fast-Food"]

data = {
    "Person_ID": range(1, n_samples+1),
    "Age": np.random.randint(18, 70, n_samples), # age between 18-70
    "Gender": np.random.choice(["Male", "Female"], n_samples), # gender
    "Height_cm": np.random.normal(170, 10, n_samples).astype(int), # height ~ N(170,10)
    "Weight_kg": np.random.normal(70, 15, n_samples).astype(int), # weight ~ N(70,15)
    "Job": np.random.choice(jobs, n_samples),
    "Area": np.random.choice(areas, n_samples),
    "Diet": np.random.choice(diets, n_samples),
    "Exercise_Hours_per_Week": np.random.randint(0, 15, n_samples),
    "Sleep_Hours_per_Day": np.random.randint(4, 10, n_samples),
    "Smoking": np.random.choice(["Yes", "No"], n_samples, p=[0.2, 0.8]),
    "Alcohol": np.random.choice(["Yes", "No"], n_samples, p=[0.3, 0.7]),
    "Cholesterol_Level": np.random.randint(150, 280, n_samples),
    "Blood_Pressure": np.random.randint(90, 160, n_samples),
    "Diabetes": np.random.choice(["Yes", "No"], n_samples, p=[0.1, 0.9]),
    "Daily_Calories": np.random.randint(1500, 4000, n_samples),
    "Steps_per_Day": np.random.randint(1000, 20000, n_samples),
    "Screen_Time_Hours": np.random.randint(1, 12, n_samples),
    "Stress_Level": np.random.randint(1, 10, n_samples), # 1 = low, 10 = high
    "Family_History": np.random.choice(["Yes", "No"], n_samples, p=[0.3, 0.7])
}

# 4. Convert dictionary → DataFrame
df = pd.DataFrame(data)
```

```
# 5. Save dataset to CSV
output_path = "D:/lecture/PTIT/20251/IntSys/data_4.2.csv"
df.to_csv(output_path, index=False)

print(f"Dataset created and saved at {output_path}")
print(df.head())
```

```
Dataset created and saved at D:/lecture/PTIT/20251/IntSys/data_4.2.csv
Person_ID Age Gender Height_cm Weight_kg Job Area \
0 1 56 Male 166 57 Teacher Rural
1 2 69 Female 163 57 IT Specialist Suburban
2 3 46 Female 165 66 Student Urban
3 4 32 Female 179 62 Nurse Rural
4 5 60 Male 177 69 IT Specialist Rural

Diet Exercise_Hours_per_Week Sleep_Hours_per_Day Smoking Alcohol \
0 Fast-Food 14 5 No Yes
1 Fast-Food 2 8 No Yes
2 Keto 14 7 No No
3 Fast-Food 5 8 No Yes
4 Vegetarian 8 6 Yes Yes

Cholesterol_Level Blood_Pressure Diabetes Daily_Calories Steps_per_Day \
0 162 134 No 2685 5204
1 218 141 No 2991 6535
2 232 114 No 2730 18035
3 216 102 No 2045 18857
4 157 106 No 3575 17431

Screen_Time_Hours Stress_Level Family_History
0 10 2 No
1 8 8 Yes
2 5 4 No
3 5 6 No
4 5 7 No
```

1. Folder Creation

```
1. os.makedirs("D:/lecture/PTIT/20251/IntSys", exist_ok=True)
2.
```

→ Creates the folder C:\DATA if it doesn't exist, so the file can be saved safely.

2. Data Size

```
1. n_samples = 10000
2.
```

→ We generate 10,000 persons.

3. Feature Generation

- **Age:** Random integer between 18–70.
- **Gender:** Randomly Male/Female.
- **Height & Weight:** Generated with a normal distribution (closer to real data).
- **Job, Area, Diet:** Chosen from predefined lists.
- **Lifestyle & Health Features:** Smoking, alcohol, sleep hours, exercise, etc.
- **Health Risks:** Cholesterol, blood pressure, diabetes.
- Total = 20 features.

4. Save as CSV

```
1. df.to_csv("D:/lecture/PTIT/20251/IntSys/data_4.2.csv", index=False)
```

2.

→ Saves dataset in the required path.

5. Preview

df.head() prints the first 5 rows for verification.

b.

```
[3]: # Nguyễn Hoàng Tùng

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Load dataset
df = pd.read_csv("D:/lecture/PTIT/20251/IntSys/data_4.2.csv")

# 2. Create BMI column
df["BMI"] = df["Weight_kg"] / ((df["Height_cm"]/100) ** 2)

# 3. Plot Age distribution
plt.figure(figsize=(8,5))
sns.histplot(df["Age"], bins=30, kde=True, color="blue")
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Count")
plt.show()

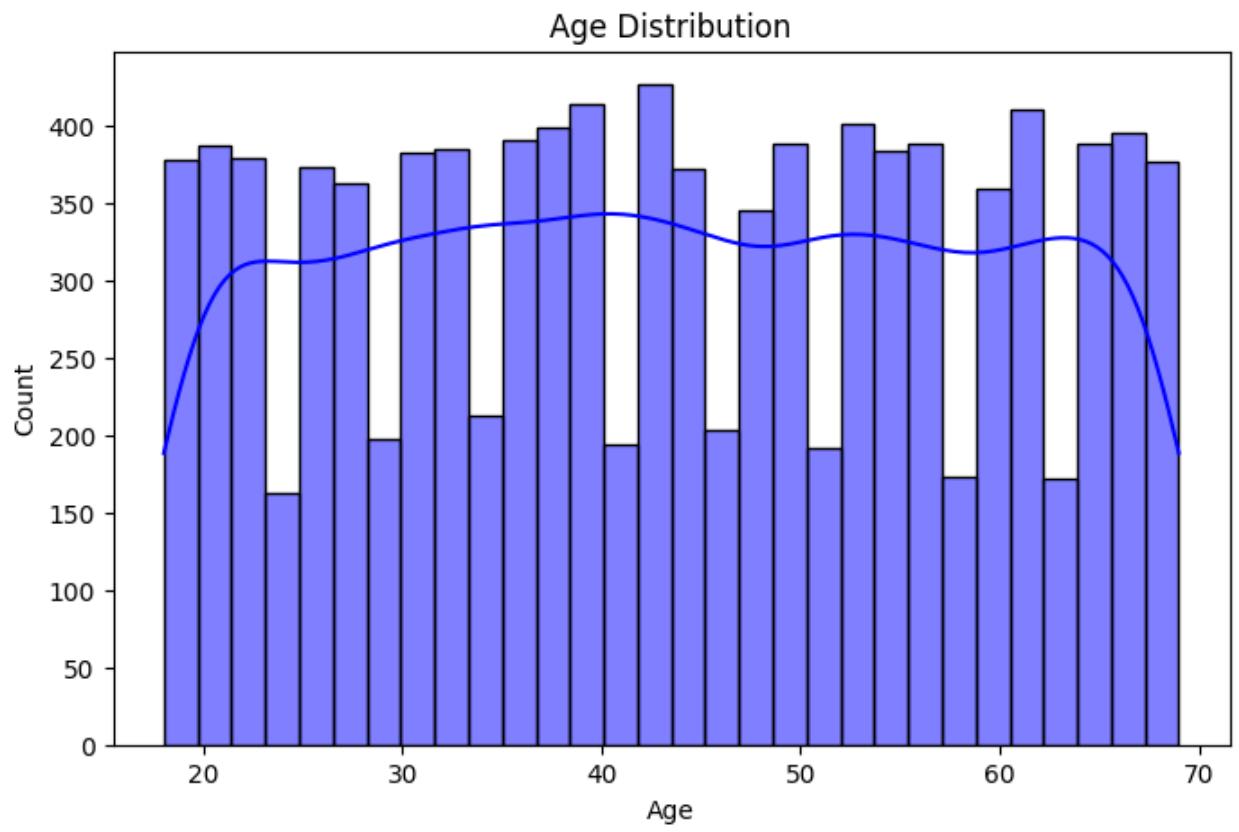
# 4. Plot Gender distribution
plt.figure(figsize=(6,5))
sns.countplot(x="Gender", data=df, palette="Set2")
plt.title("Gender Distribution")
plt.show()

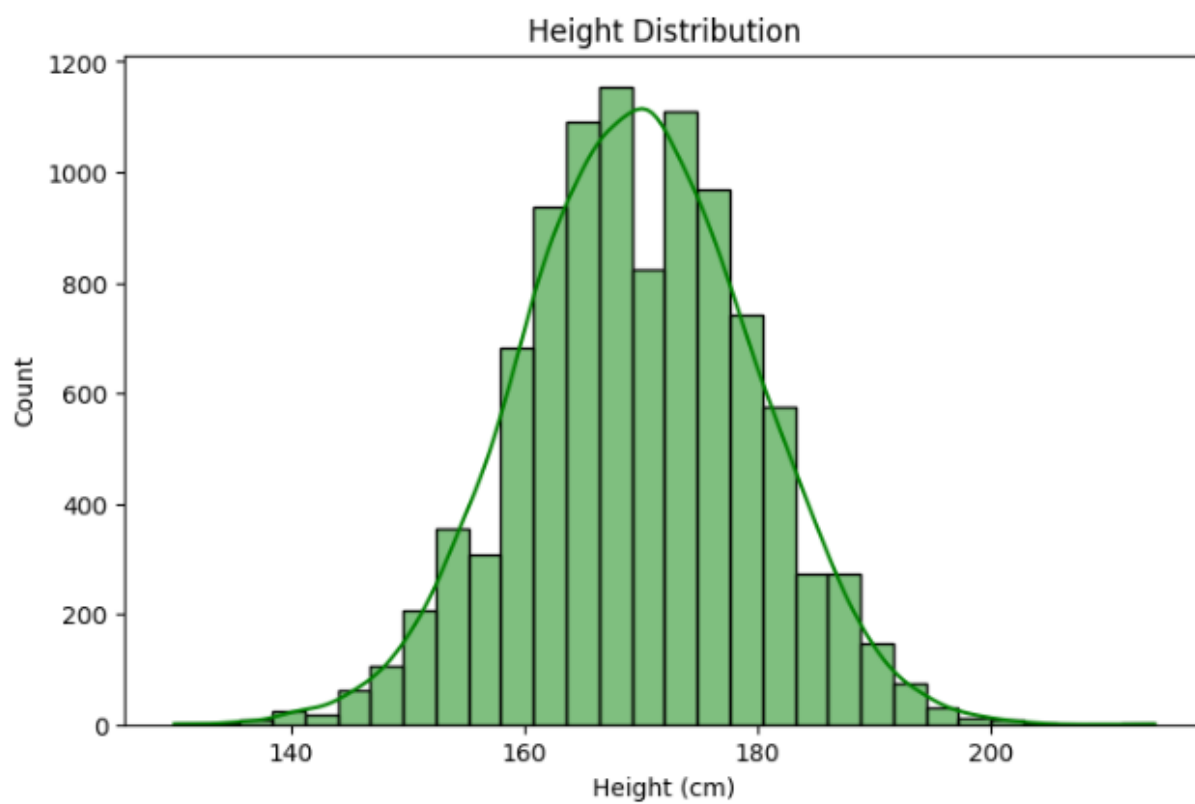
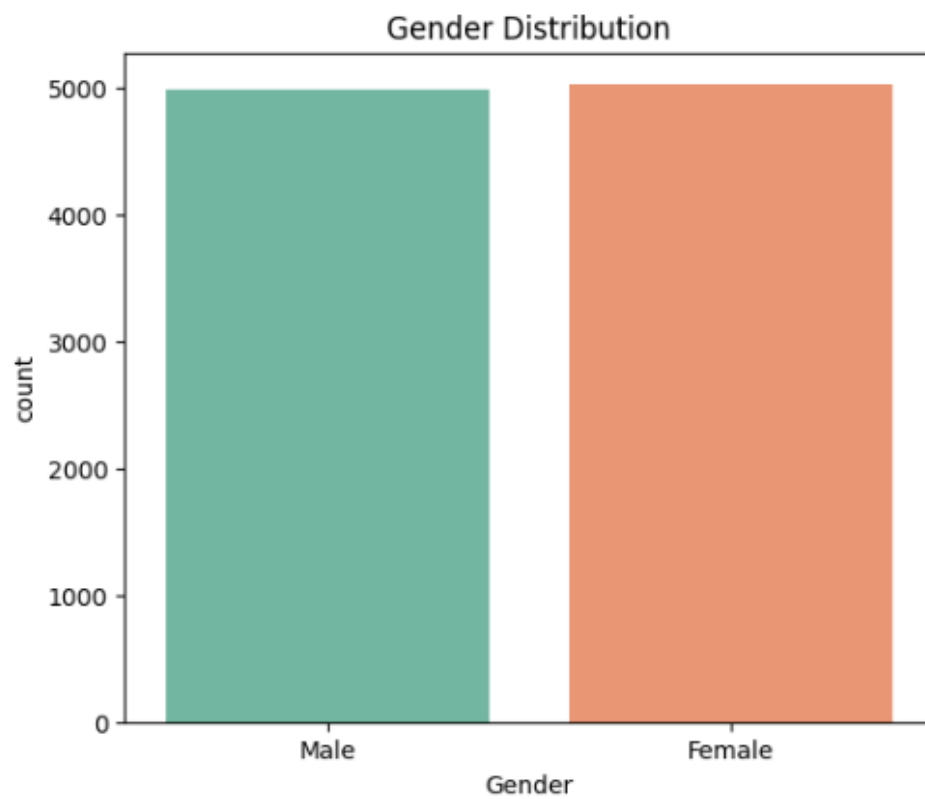
# 5. Plot Height distribution
plt.figure(figsize=(8,5))
sns.histplot(df["Height_cm"], bins=30, kde=True, color="green")
plt.title("Height Distribution")
plt.xlabel("Height (cm)")
plt.show()

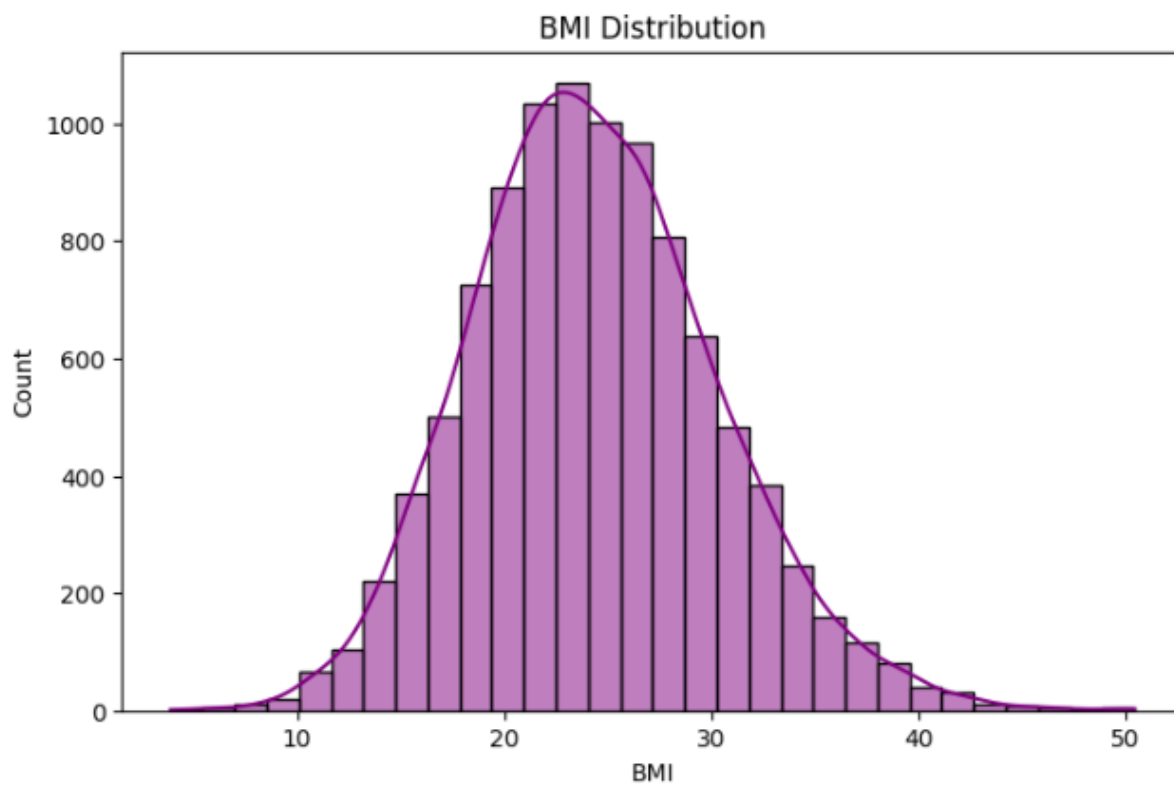
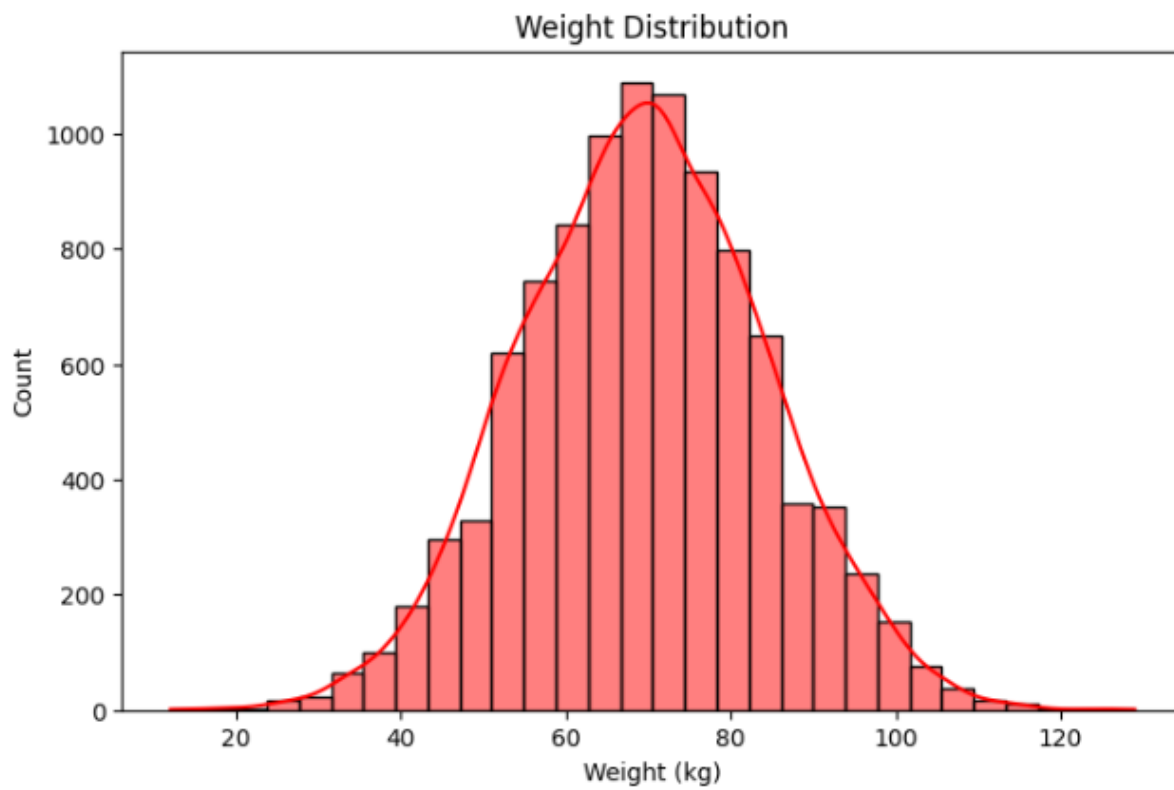
# 6. Plot Weight distribution
plt.figure(figsize=(8,5))
sns.histplot(df["Weight_kg"], bins=30, kde=True, color="red")
plt.title("Weight Distribution")
plt.xlabel("Weight (kg)")
plt.show()

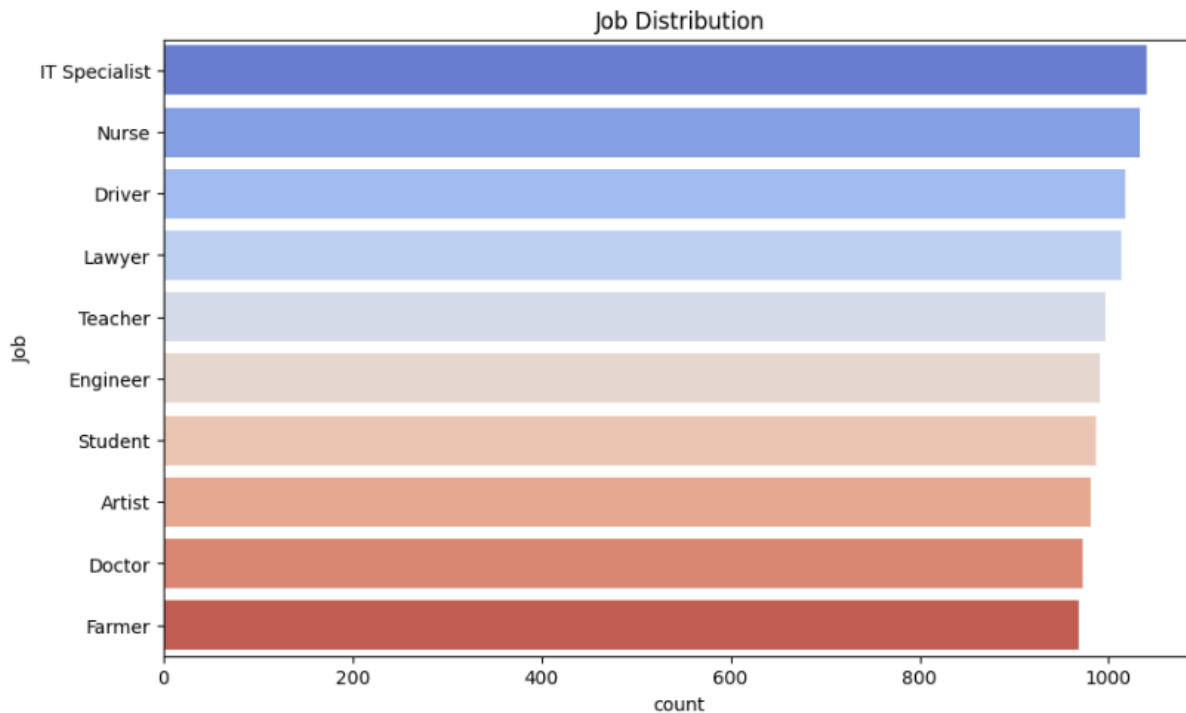
# 7. Plot BMI distribution
plt.figure(figsize=(8,5))
sns.histplot(df["BMI"], bins=30, kde=True, color="purple")
plt.title("BMI Distribution")
plt.xlabel("BMI")
plt.show()

# 8. Plot Job distribution
plt.figure(figsize=(10,6))
sns.countplot(y="Job", data=df, order=df["Job"].value_counts().index, palette="coolwarm")
plt.title("Job Distribution")
plt.show()
```









1. Load dataset

```
1. df = pd.read_csv("D:/lecture/PTIT/20251/IntSys/data_4.2.csv")
2.
```

→ Reads the dataset created in part (a).

2. BMI Calculation

```
1. df["BMI"] = df["Weight_kg"] / ((df["Height_cm"]/100) ** 2)
2.
```

→ Formula: $\text{weight (kg)} / \text{height}^2 (\text{m}^2)$

3. Distribution Plots

- **Age:** Histogram + KDE (shows normal distribution shape).
- **Gender:** Countplot (Male vs Female).
- **Height & Weight:** Histograms with normal distribution curves.
- **BMI:** Histogram to see underweight, normal, overweight distribution.
- **Job:** Countplot to show which job has more records.

C.

```
[5]: # Nguyễn Hoàng Tùng

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, classification_report
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

# 1. Load dataset
df = pd.read_csv("D:/lecture/PTIT/20251/IntSys/data_4.2.csv")

# 2. Compute BMI
df["BMI"] = df["Weight_kg"] / ((df["Height_cm"]/100) ** 2)

# 3. Create BMI Category
def bmi_category(bmi):
    if bmi < 18.5:
        return "Underweight"
    elif bmi < 25:
        return "Normal"
    else:
        return "Overweight"

df["BMI_Category"] = df["BMI"].apply(bmi_category)

# 4. Encode categorical features
categorical_cols = ["Gender", "Job", "Area", "Diet", "Smoking", "Alcohol", "Diabetes", "Family_History"]
encoder = LabelEncoder()
for col in categorical_cols:
    df[col] = encoder.fit_transform(df[col])

# 5. Select features (exclude Person_ID, BMI, BMI_Category)
X = df.drop(columns=["Person_ID", "BMI", "BMI_Category"])
y = df["BMI_Category"]

# 6. Scale numerical features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 7. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42, stratify=y)

# 8. Define models
models = {
    "Logistic Regression": LogisticRegression(max_iter=500),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(n_estimators=100),
    "SVM": SVC(),
    "KNN": KNeighborsClassifier(n_neighbors=5)
}

# 9. Train & Evaluate
results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    results[name] = acc
    print(f"\n{name} Accuracy: {acc:.4f}")
    print(classification_report(y_test, y_pred))
```

```

Logistic Regression Accuracy: 0.9870
precision    recall  f1-score   support

   Normal      0.98      0.99      0.98      813
  Overweight      0.99      0.99      0.99      871
  Underweight      0.98      0.97      0.98      316

 accuracy      0.99      0.99      0.99      2000
 macro avg      0.99      0.98      0.98      2000
 weighted avg      0.99      0.99      0.99      2000


Decision Tree Accuracy: 0.9850
precision    recall  f1-score   support

   Normal      0.98      0.98      0.98      813
  Overweight      1.00      1.00      1.00      871
  Underweight      0.96      0.97      0.96      316

 accuracy      0.98      0.98      0.98      2000
 macro avg      0.98      0.98      0.98      2000
 weighted avg      0.99      0.98      0.99      2000


Random Forest Accuracy: 0.9745
precision    recall  f1-score   support

   Normal      0.95      0.99      0.97      813
  Overweight      0.99      0.99      0.99      871
  Underweight      0.99      0.90      0.94      316

 accuracy      0.98      0.96      0.97      2000
 macro avg      0.98      0.96      0.97      2000
 weighted avg      0.98      0.97      0.97      2000


SVM Accuracy: 0.9530
precision    recall  f1-score   support

   Normal      0.93      0.96      0.94      813
  Overweight      0.98      0.98      0.98      871
  Underweight      0.96      0.87      0.91      316

 accuracy      0.95      0.94      0.95      2000
 macro avg      0.95      0.94      0.94      2000
 weighted avg      0.95      0.95      0.95      2000


KNN Accuracy: 0.6855
precision    recall  f1-score   support

   Normal      0.59      0.75      0.66      813
  Overweight      0.78      0.76      0.77      871
  Underweight      0.82      0.31      0.45      316

 accuracy      0.73      0.61      0.69      2000
 macro avg      0.73      0.61      0.63      2000
 weighted avg      0.71      0.69      0.68      2000

```

1. BMI Category

- Converts BMI value → label (Underweight, Normal, Overweight).

2. Encoding

- Converts categorical features (Gender, Job, Area, etc.) into numeric values.

3. Feature Scaling

- Normalizes numerical values for better performance (especially SVM & KNN).

4. Models

- Trains **5 classifiers** on 80% training data, tests on 20%.

5. Evaluation

- Prints **accuracy** and **classification report** (precision, recall, F1-score).

d.

```
[6]: # Nguyễn Hoàng Tùng

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Conv1D, Flatten, SimpleRNN, LSTM, Embedding
from tensorflow.keras.utils import to_categorical

# 1. Load dataset
df = pd.read_csv("D:/lecture/PTIT/20251/IntSys/data_4.2.csv")

# 2. Compute BMI & categories
df["BMI"] = df["Weight_kg"] / ((df["Height_cm"]/100) ** 2)
def bmi_category(bmi):
    if bmi < 18.5:
        return "Underweight"
    elif bmi < 25:
        return "Normal"
    else:
        return "Overweight"
df["BMI_Category"] = df["BMI"].apply(bmi_category)

# 3. Encode categorical features
categorical_cols = ["Gender", "Job", "Area", "Diet", "Smoking", "Alcohol", "Diabetes", "Family_History"]
encoder = LabelEncoder()
for col in categorical_cols:
    df[col] = encoder.fit_transform(df[col])

# 4. Prepare features & labels
X = df.drop(columns=["Person_ID", "BMI", "BMI_Category"])
y = df["BMI_Category"]

# Encode labels
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y) # 0=Normal, 1=Overweight, 2=Underweight
y_categorical = to_categorical(y_encoded, num_classes=3)

# Scale X
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Reshape for CNN/RNN/LSTM (samples, timesteps, features)
X_resaped = X_scaled.reshape(X_scaled.shape[0], X_scaled.shape[1], 1)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X_resaped, y_categorical, test_size=0.2, random_state=42, stratify=y_encoded)
```

```

# =====
# 5. Build CNN Model
# =====
cnn_model = Sequential([
    Conv1D(32, kernel_size=3, activation='relu', input_shape=(X_train.shape[1],1)),
    Conv1D(64, kernel_size=3, activation='relu'),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dense(3, activation='softmax')
])
cnn_model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])

# =====
# 6. Build RNN Model
# =====
rnn_model = Sequential([
    SimpleRNN(64, activation='relu', return_sequences=True, input_shape=(X_train.shape[1],1)),
    Dropout(0.3),
    SimpleRNN(32, activation='relu', return_sequences=True),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(3, activation='softmax')
])
rnn_model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])

# =====
# 7. Build LSTM Model
# =====
lstm_model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(X_train.shape[1],1)),
    Dropout(0.3),
    LSTM(32, return_sequences=True),
    LSTM(16),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(3, activation='softmax')
])
lstm_model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])

# =====
# 8. Train models
# =====
print("\nTraining CNN...")
cnn_model.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.2, verbose=1)

print("\nTraining RNN...")
rnn_model.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.2, verbose=1)

print("\nTraining LSTM...")
lstm_model.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.2, verbose=1)

# =====
# 9. Evaluate
# =====
cnn_acc = cnn_model.evaluate(X_test, y_test, verbose=0)[1]
rnn_acc = rnn_model.evaluate(X_test, y_test, verbose=0)[1]
lstm_acc = lstm_model.evaluate(X_test, y_test, verbose=0)[1]

print(f"\nCNN Accuracy: {cnn_acc:.4f}")
print(f"\nRNN Accuracy: {rnn_acc:.4f}")
print(f"\nLSTM Accuracy: {lstm_acc:.4f}")

```

```

Training CNN...
Epoch 1/10
100/100 ————— 2s 9ms/step - accuracy: 0.8100 - loss: 0.4379 - val_accuracy: 0.9275 - val_loss: 0.1735
Epoch 2/10
100/100 ————— 1s 5ms/step - accuracy: 0.9366 - loss: 0.1503 - val_accuracy: 0.9488 - val_loss: 0.1235
Epoch 3/10
100/100 ————— 1s 5ms/step - accuracy: 0.9447 - loss: 0.1302 - val_accuracy: 0.9581 - val_loss: 0.0935
Epoch 4/10
100/100 ————— 1s 8ms/step - accuracy: 0.9605 - loss: 0.1028 - val_accuracy: 0.9600 - val_loss: 0.0932
Epoch 5/10
100/100 ————— 1s 6ms/step - accuracy: 0.9561 - loss: 0.1050 - val_accuracy: 0.9650 - val_loss: 0.0743
Epoch 6/10
100/100 ————— 1s 7ms/step - accuracy: 0.9641 - loss: 0.0829 - val_accuracy: 0.9725 - val_loss: 0.0628
Epoch 7/10
100/100 ————— 1s 7ms/step - accuracy: 0.9608 - loss: 0.0931 - val_accuracy: 0.9563 - val_loss: 0.0888
Epoch 8/10
100/100 ————— 1s 7ms/step - accuracy: 0.9641 - loss: 0.0845 - val_accuracy: 0.9688 - val_loss: 0.0672
Epoch 9/10
100/100 ————— 1s 6ms/step - accuracy: 0.9702 - loss: 0.0687 - val_accuracy: 0.9737 - val_loss: 0.0607
Epoch 10/10
100/100 ————— 1s 6ms/step - accuracy: 0.9733 - loss: 0.0640 - val_accuracy: 0.9725 - val_loss: 0.0582

Training RNN...
Epoch 1/10
100/100 ————— 4s 16ms/step - accuracy: 0.7628 - loss: 0.5134 - val_accuracy: 0.9031 - val_loss: 0.2167
Epoch 2/10
100/100 ————— 2s 12ms/step - accuracy: 0.9030 - loss: 0.2234 - val_accuracy: 0.9438 - val_loss: 0.1507
Epoch 3/10
100/100 ————— 1s 9ms/step - accuracy: 0.9180 - loss: 0.1923 - val_accuracy: 0.9556 - val_loss: 0.1172
Epoch 4/10
100/100 ————— 2s 12ms/step - accuracy: 0.9333 - loss: 0.1614 - val_accuracy: 0.9563 - val_loss: 0.1085
Epoch 5/10
100/100 ————— 1s 9ms/step - accuracy: 0.9358 - loss: 0.1502 - val_accuracy: 0.9212 - val_loss: 0.1588
Epoch 6/10
100/100 ————— 1s 10ms/step - accuracy: 0.9427 - loss: 0.1444 - val_accuracy: 0.9538 - val_loss: 0.1118
Epoch 7/10
100/100 ————— 1s 9ms/step - accuracy: 0.9434 - loss: 0.1378 - val_accuracy: 0.9606 - val_loss: 0.0988
Epoch 8/10
100/100 ————— 2s 12ms/step - accuracy: 0.9480 - loss: 0.1218 - val_accuracy: 0.9619 - val_loss: 0.0936
Epoch 9/10
100/100 ————— 1s 11ms/step - accuracy: 0.9502 - loss: 0.1184 - val_accuracy: 0.9613 - val_loss: 0.1049
Epoch 10/10
100/100 ————— 1s 13ms/step - accuracy: 0.9527 - loss: 0.1138 - val_accuracy: 0.9581 - val_loss: 0.0964

Training LSTM...
Epoch 1/10
100/100 ————— 7s 28ms/step - accuracy: 0.4244 - loss: 1.0277 - val_accuracy: 0.4481 - val_loss: 1.0042
Epoch 2/10
100/100 ————— 5s 23ms/step - accuracy: 0.4236 - loss: 1.0154 - val_accuracy: 0.4044 - val_loss: 1.0043
Epoch 3/10
100/100 ————— 3s 22ms/step - accuracy: 0.4431 - loss: 1.0001 - val_accuracy: 0.4781 - val_loss: 0.9613
Epoch 4/10
100/100 ————— 2s 24ms/step - accuracy: 0.4725 - loss: 0.9670 - val_accuracy: 0.4863 - val_loss: 0.9581
Epoch 5/10
100/100 ————— 3s 24ms/step - accuracy: 0.4773 - loss: 0.9492 - val_accuracy: 0.4850 - val_loss: 0.9376
Epoch 6/10
100/100 ————— 3s 24ms/step - accuracy: 0.4981 - loss: 0.9315 - val_accuracy: 0.4919 - val_loss: 0.9190
Epoch 7/10
100/100 ————— 2s 22ms/step - accuracy: 0.4972 - loss: 0.9238 - val_accuracy: 0.4806 - val_loss: 0.9188
Epoch 8/10
100/100 ————— 3s 25ms/step - accuracy: 0.5116 - loss: 0.9038 - val_accuracy: 0.4825 - val_loss: 0.9149
Epoch 9/10
100/100 ————— 2s 25ms/step - accuracy: 0.5198 - loss: 0.8938 - val_accuracy: 0.5100 - val_loss: 0.9149
Epoch 10/10
100/100 ————— 3s 24ms/step - accuracy: 0.5303 - loss: 0.8723 - val_accuracy: 0.5125 - val_loss: 0.9213

CNN Accuracy: 0.9785
RNN Accuracy: 0.9565
LSTM Accuracy: 0.5270

```

1. Reshape for Sequential Models

- CNN, RNN, LSTM need 3D input → (samples, timesteps, features).
- We treat each feature as a “time step”.

2. CNN Model

- Uses **Conv1D layers** to detect feature patterns.
- Flatten → Dense layers for classification.

3. RNN Model

- Uses **SimpleRNN layers** to learn sequential dependencies.

4. LSTM Model

- Uses **LSTM layers** for long-term dependencies.
- Deeper network with multiple LSTM layers + Dense layers.

5. Training

- 10 epochs, batch size = 64, 20% validation split.
- Evaluates accuracy on test set.

e.

```
[9]: # Nguyễn Hoàng Tùng

from sklearn.metrics import mean_absolute_error, mean_squared_error, accuracy_score
import numpy as np
import pandas as pd

# =====
# 0. Ensure we have both datasets
# =====
# X_scaled, y (string labels) -> for ML models
# X_rescaled, y_categorical, y_encoded -> for DL models

X_train_ml, X_test_ml, y_train_ml, y_test_ml = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# For DL we already have: X_train, X_test, y_train, y_test (one-hot encoded)

y_test_labels = np.argmax(y_test, axis=1) # integer labels for DL

# =====
# Helper function
# =====
def evaluate_model(name, y_true, y_pred):
    # Convert labels if necessary
    if isinstance(y_true[0], str): # sklearn models
        acc = accuracy_score(y_true, y_pred)
        mae = mean_absolute_error(
            LabelEncoder().fit_transform(y_true),
            LabelEncoder().fit_transform(y_pred)
        )
        mse = mean_squared_error(
            LabelEncoder().fit_transform(y_true),
            LabelEncoder().fit_transform(y_pred)
        )
    else: # deep learning integer labels
        acc = accuracy_score(y_true, y_pred)
        mae = mean_absolute_error(y_true, y_pred)
        mse = mean_squared_error(y_true, y_pred)

    rmse = np.sqrt(mse)
    return {"Model": name, "Accuracy": acc, "MAE": mae, "MSE": mse, "RMSE": rmse}

results_list = []

# =====
# 1. Evaluate Basic ML Models (2D data)
# =====
for name, model in models.items():
    y_pred = model.predict(X_test_ml)
    results_list.append(evaluate_model(name, y_test_ml.tolist(), y_pred.tolist()))
```

```

# =====
# 2. Evaluate Deep Learning Models (3D data)
# =====
cnn_pred = np.argmax(cnn_model.predict(X_test), axis=1)
rnn_pred = np.argmax(rnn_model.predict(X_test), axis=1)
lstm_pred = np.argmax(lstm_model.predict(X_test), axis=1)

results_list.append(evaluate_model("CNN", y_test_labels, cnn_pred))
results_list.append(evaluate_model("RNN", y_test_labels, rnn_pred))
results_list.append(evaluate_model("LSTM", y_test_labels, lstm_pred))

# =====
# 3. Create Comparison Table
# =====
results_df = pd.DataFrame(results_list)
print("\n📊 Model Evaluation Results:")
print(results_df)

```

```

63/63 ————— 0s 2ms/step
63/63 ————— 1s 7ms/step
63/63 ————— 1s 8ms/step

```

```

📊 Model Evaluation Results:

```

	Model	Accuracy	MAE	MSE	RMSE
0	Logistic Regression	0.9870	0.0205	0.0355	0.188414
1	Decision Tree	0.9850	0.0265	0.0495	0.222486
2	Random Forest	0.9745	0.0430	0.0780	0.279285
3	SVM	0.9530	0.0735	0.1265	0.355668
4	KNN	0.6855	0.4300	0.6610	0.813019
5	CNN	0.9785	0.0335	0.0575	0.239792
6	RNN	0.9565	0.0680	0.1170	0.342053
7	LSTM	0.5270	0.5360	0.6620	0.813634

1. evaluate_model() function

- Computes Accuracy, MAE, MSE, RMSE for any model.

2. Basic ML Models

- Predictions are already in label format.

3. Deep Learning Models

- Predictions are **probabilities** → use np.argmax() to convert to class index.
- Compare with y_test_labels.

4. Results Table

- Collects all metrics in a single Pandas DataFrame for easy comparison.

f.

```
[10]: # Nguyễn Hoàng Tùng
```

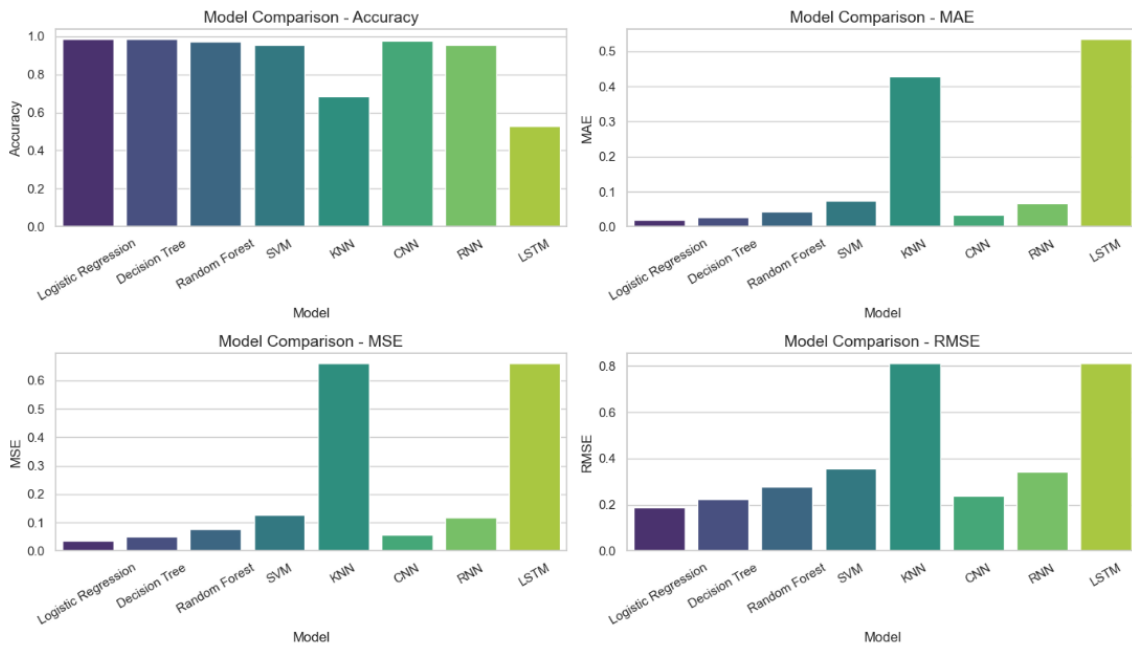
```
import matplotlib.pyplot as plt
import seaborn as sns

# Ensure we use the results from part (e)
metrics = ["Accuracy", "MAE", "MSE", "RMSE"]

# Set plot style
sns.set(style="whitegrid")
plt.figure(figsize=(14, 8))

# Plot each metric
for i, metric in enumerate(metrics, 1):
    plt.subplot(2, 2, i)
    sns.barplot(x="Model", y=metric, data=results_df, palette="viridis")
    plt.title(f"Model Comparison - {metric}", fontsize=14)
    plt.xticks(rotation=30)
    plt.ylabel(metric)

plt.tight_layout()
plt.show()
```



- Four bar charts:
 - Accuracy comparison of all models
 - MAE comparison
 - MSE comparison
 - RMSE comparison

This gives you a **side-by-side overview** of how Logistic Regression, SVM, Random Forest, KNN, CNN, RNN, LSTM perform.

g.

```
[*]: # Nguyễn Hoàng Tùng

from flask import Flask, request, render_template_string
import numpy as np
import pandas as pd
import joblib
from tensorflow.keras.models import load_model

# -----
# 1. Load dataset to get exact column names
# -----
df = pd.read_csv(r"D:/lecture/PTIT/20251/IntSys/data_4.2.csv")
print("✅ Columns in dataset:", df.columns.tolist())

# Detect label column automatically (last column assumed as target)
TARGET_COL = df.columns[-1]
RAW_COLUMNS = df.drop(TARGET_COL, axis=1).columns.tolist()
print("✅ Using feature columns:", RAW_COLUMNS)
print("✅ Target column:", TARGET_COL)

# -----
# 2. Load best model
# -----
try:
    if best_model_name in ["CNN", "RNN", "LSTM"]:
        model = load_model("best_model.h5")
    else:
        model = joblib.load("best_model.pkl")
except Exception as e:
    print("⚠️ Could not load model:", e)

# -----
# 3. Define labels (adjust if your dataset has more/different classes)
# -----
labels = df[TARGET_COL].unique().tolist()
print("✅ Labels:", labels)

# -----
# 4. Flask app setup
# -----
app = Flask(__name__)

def predict_health_status(features):
    # Create DataFrame with same columns as training data
    features_df = pd.DataFrame([features], columns=RAW_COLUMNS)

    if best_model_name in ["CNN", "RNN", "LSTM"]:
        features_3d = features_df.values.reshape(features_df.shape[0], features_df.shape[1], 1)
        pred = np.argmax(model.predict(features_3d), axis=1)[0]
    else:
        pred = model.predict(features_df.values)
        # Handle sklearn vs keras output
        if hasattr(pred, "argmax"):
            pred = pred.argmax(axis=1)[0]
        else:
            pred = int(pred[0])
    return labels[pred]
```

```
# -----
# 5. Simple HTML UI
# -----
HTML_PAGE = """
<!DOCTYPE html>
<html>
<head>
  <title>Health Prediction</title>
  <style>
    body { font-family: Arial; margin: 20px; }
    .container { width: 500px; margin: auto; }
    label { display: block; margin-top: 10px; }
    input { width: 100%; padding: 8px; margin-top: 5px; }
    button { margin-top: 15px; padding: 10px; width: 100%; }
    .result { margin-top: 20px; font-weight: bold; color: green; }
  </style>
</head>
<body>
<div class="container">
  <h2> 🏥 Health Status Prediction</h2>
  <form method="POST">
    {% for col in columns %}
    <label>{{col}}</label>
    <input type="text" name="{{col}}" required>
    {% endfor %}
    <button type="submit">Predict</button>
  </form>
  {% if result %}
  <div class="result">✅ Prediction: {{ result }}</div>
  {% endif %}
</div>
</body>
</html>
"""
```

```
@app.route("/", methods=["GET", "POST"])
def index():
    result = None
    if request.method == "POST":
        inputs = []
        for col in RAW_COLUMNS:
            val = request.form[col]
            try:
                inputs.append(float(val))
            except:
                inputs.append(val) # keep string for categorical
        result = predict_health_status(inputs)
    return render_template_string(HTML_PAGE, columns=RAW_COLUMNS, result=result)

# -----
# 6. Run Flask inside Jupyter
# -----
from werkzeug.serving import run_simple

print("🚀 Starting Flask app... Visit http://127.0.0.1:5000 in your browser.")
run_simple('localhost', 5000, app)
```

```
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://localhost:5000
Press CTRL+C to quit
✅ Columns in dataset: ['Person_ID', 'Age', 'Gender', 'Height_cm', 'Weight_kg', 'Job', 'Area', 'Diet', 'Exercise_Hours_per_Week', 'Sleep_Hours_per_Day', 'Smoking', 'Alcohol', 'Cholesterol_Level', 'Blood_Pressure', 'Diabetes', 'Daily_Calories', 'Steps_per_Day', 'Screen_Time_Hours', 'Stress_Level', 'Family_History']
✅ Using feature columns: ['Person_ID', 'Age', 'Gender', 'Height_cm', 'Weight_kg', 'Job', 'Area', 'Diet', 'Exercise_Hours_per_Week', 'Sleep_Hours_per_Day', 'Smoking', 'Alcohol', 'Cholesterol_Level', 'Blood_Pressure', 'Diabetes', 'Daily_Calories', 'Steps_per_Day', 'Screen_Time_Hours', 'Stress_Level']
✅ Target column: Family_History
✅ Labels: ['No', 'Yes']
🚀 Starting Flask app... Visit http://127.0.0.1:5000 in your browser.
127.0.0.1 - - [14/Sep/2025 00:03:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2025 00:03:25] "GET /favicon.ico HTTP/1.1" 404 -
```

1. **Import Libraries** – Flask for web server, pandas/numpy for data, joblib/Keras for loading models.
2. **Load Dataset & Columns** – Reload CSV → get feature names (RAW_COLUMNS) and target column (e.g. "BMI_Class").
3. **Load Model** – Use .pkl if sklearn, .h5 if deep learning.

4. **Define Labels** – Extract class labels from dataset (e.g. "Underweight", "Normal", "Overweight").
5. **Prediction Function** – Convert form inputs → DataFrame → reshape if needed → call model.predict() → return label.
6. **HTML Form** – Simple web page with input fields for all features.
7. **Flask Route** – GET shows form, POST runs prediction and displays result.
8. **Run App** – Start Flask in Jupyter, test via browser at <http://127.0.0.1:5000/>.

h + i.

```
[1]: # Nguyễn Hoàng Tùng

import json
import random
import math
from itertools import product
from datetime import datetime

random.seed(42)

OUTPATH = r"D:\lecture\PTIT\20251\IntSys\kb_healthGuide.json"

# --- Configuration: tune these to change KB size ---
# small sections
n_bmi_steps = 301 # BMI from 10.0 to 40.0 by 0.1 -> 301 entries
n_waist_entries = 200 # waist rules by age/gender groups
diseases = [
    {
        "name": "Type 2 Diabetes",
        "risk_factors": ["BMI >= 25", "waist > threshold", "family_history"],
        "diet_recommendations": ["Limit simple carbs", "Choose whole grains"],
        "exercise_recommendations": ["Brisk walk 30 min daily"]
    },
    {
        "name": "Hypertension",
        "diet_recommendations": ["DASH diet", "Limit sodium <5g/day"],
        "exercise_recommendations": ["Moderate cardio 30 min, most days"]
    },
    {
        "name": "Hyperlipidemia",
        "diet_recommendations": ["Limit saturated fat", "Increase fiber"],
        "exercise_recommendations": ["Aerobic exercise 30-60 min"]
    }
] # small list

n_food_items = 5000 # synthetic food list size
n_exercises = 300 # exercise entries
```

```

# big sections (this will be the main driver to reach 3M)
n_mealplans = 120000 # number of generated meal plans
n_rules_target = 3000000 - (n_bmi_steps + n_waist_entries + len(diseases) + n_food_items + n_mealplans + n_exercises)
# ensure non-negative
if n_rules_target < 0:
    n_rules_target = 1_000_000

# --- Utility generators ---
def gen_bmi_classification():
    """
    Yield BMI classification entries with fine-grained thresholds (0.1 steps).
    """
    for i in range(n_bmi_steps):
        bmi = round(10.0 + 0.1 * i, 1)
        if bmi < 18.5:
            label = "Underweight"
            advice = ["Increase calorie intake with healthy foods", "Resistance training 2-3x/week"]
        elif bmi < 25:
            label = "Normal"
            advice = ["Maintain balanced diet", "150 min moderate exercise/week"]
        elif bmi < 30:
            label = "Overweight"
            advice = ["Reduce calorie intake moderately", "Increase aerobic activity"]
        else:
            label = "Obese"
            advice = ["Medical evaluation", "Structured weight loss program"]
        yield {"bmi": bmi, "category": label, "advice": advice}

```

```

def gen_waist_risk():
    """
    Waist risk entries stratified by age and gender.
    """
    age_groups = [(18,30),(31,40),(41,50),(51,60),(61,80)]
    genders = ["Male","Female"]
    for gender in genders:
        for (a,b) in age_groups:
            # threshold example
            threshold = 94 if gender=="Male" else 80
            yield {
                "gender": gender,
                "age_range": f"{a}-{b}",
                "waist_threshold_cm": threshold,
                "risk": "Increased risk if waist > threshold",
                "advice": ["Measure waist at navel", "Aim to reduce abdominal fat"]
            }

def gen_food_items():
    """
    Create synthetic food items; ideally one would import USDA data but we synthesize.
    """
    categories = ["Grains","Vegetable","Fruit","Meat","Fish","Dairy","Nuts","Beverage","Snack"]
    grain_names = ["rice","bread","oats","quinoa","pasta","barley"]
    vega = ["spinach","broccoli","carrot","tomato","pepper","cucumber"]
    fruits = ["apple","banana","orange","berries","mango","pear"]
    meats = ["chicken","beef","pork","lamb","turkey"]
    fish = ["salmon","tuna","mackerel","cod"]
    dairy = ["milk","yogurt","cheese"]
    nuts = ["almond","walnut","peanut"]
    beverages = ["coffee","tea","soda","juice"]
    snacks = ["chips","cookie","chocolate"]
    pools = [grain_names, vega, fruits, meats, fish, dairy, nuts, beverages, snacks]

```

```

created = 0
idx = 0
while created < n_food_items:
    cat_index = idx % len(pools)
    pool = pools[cat_index]
    name = random.choice(pool)
    variant = random.choice(["plain", "roasted", "salted", "fresh", "canned", "steamed", "grilled", "baked"])
    full_name = f"{variant} {name}"
    calories = random.randint(20, 700)
    protein = round(random.uniform(0.0, 40.0), 1)
    fat = round(random.uniform(0.0, 50.0), 1)
    carbs = max(0, round((calories - protein*4 - fat*9)/4, 1))
    gly_index = random.randint(10, 95)
    advice = []
    if calories > 400:
        advice.append("High calorie - moderate portion")
    if gly_index > 70:
        advice.append("High GI - avoid for weight loss")
    yield {
        "id": f"food_{created+1}",
        "name": full_name,
        "category": categories[cat_index],
        "calories_per_100g": calories,
        "protein_g": protein,
        "fat_g": fat,
        "carbs_g": carbs,
        "glycemic_index": gly_index,
        "advice": advice
    }
    created += 1
    idx += 1

```

```

def gen_exercises():
    base = [
        ("Walking", 150), ("Running", 400), ("Cycling", 350), ("Swimming", 370),
        ("Squat", 120), ("Deadlift", 200), ("Push-ups", 100), ("Yoga", 80),
        ("HIIT", 500), ("Rowing", 420)
    ]
    # expand to n_exercises
    for i in range(n_exercises):
        name, burn = random.choice(base)
        variant = random.choice(["light", "moderate", "intense", "beginner", "advanced"])
        yield {
            "id": f"ex_{i+1}",
            "name": f"{variant} {name}",
            "calories_burned_per_30min": max(10, int(burn * random.uniform(0.6, 1.4))),
            "difficulty": random.choice(["Easy", "Moderate", "Hard"]),
            "suitable_BMI_range": f"{random.randint(15, 20)}-{random.randint(21, 40)}"
        }

def gen_meal_plans(food_ids):
    """
    Generate meal plans by combining 3-5 food items to reach calorie targets
    We'll create n_mealplans entries.
    """
    cal_targets = [1200, 1500, 1800, 2000, 2200, 2500]
    diets = ["Balanced", "Low-carb", "High-protein", "Vegetarian", "Keto", "Mediterranean"]
    created = 0
    max_food = len(food_ids)
    while created < n_mealplans:
        k = random.randint(3, 5)
        picks = [random.choice(food_ids) for _ in range(k)]
        target = random.choice(cal_targets)
        diet = random.choice(diets)
        # estimate calories by summing chosen foods' calories (we only have ids here)
        plan = {
            "id": f"mealplan_{created+1}",
            "foods": picks,
            "target_calories": target,
            "diet_type": diet,
            "advice": [f"Follow {diet} pattern", "Adjust portions to meet calorie target"]
        }
        yield plan
        created += 1

```

```

def gen_rules(bmi_step_count, age_groups, genders, diseases_list, activities, n_rules):
    """
    Generate 'n_rules' rules combinatorically.
    A rule is small JSON dict with conditions and advice.
    """
    created = 0
    # pool of possible phrase fragments
    bmi_indices = list(range(bmi_step_count)) # each index corresponds to a BMI step
    while created < n_rules:
        bmi_idx = random.choice(bmi_indices)
        age = random.choice(age_groups)
        gender = random.choice(genders)
        disease = random.choice(diseases_list)["name"]
        activity = random.choice(activities)
        # create conditions using these
        bmi_value = round(10.0 + 0.1 * bmi_idx, 1)
        cond = {
            "BMI": f"{bmi_value}",
            "age": f"{age[0]}-{age[1]}",
            "gender": gender,
            "disease": disease if random.random() < 0.15 else None,
            "activity_level": activity
        }
        advice = []
        if bmi_value >= 30:
            advice.append("High risk: recommend clinical weight loss program")
            advice.append("Reduce refined carbs and added sugars")
        elif bmi_value >= 25:
            advice.append("Reduce portion size, increase activity")
        elif bmi_value < 18.5:
            advice.append("Increase calorie-dense healthy foods")
        else:
            advice.append("Maintain healthy lifestyle")
        # possibly add exercise/diet specifics
        if "High" in activity:
            advice.append("Include resistance training 2x/week")
        rule = {
            "id": f"rule_{created+1}",
            "if": {k:v for k,v in cond.items() if v is not None},
            "then": {
                "risk": "elevated" if bmi_value>=25 else "normal",
                "advice": advice
            }
        }
        created += 1
        yield rule

```

```

# --- Now stream-write the big JSON object with arrays ---
def build_kb_json(outpath=OUTPATH):
    print("Starting KB build:", datetime.now().isoformat())
    with open(outpath, "w", encoding="utf-8") as f:
        f.write("{\n")
        # 1. BMI_Classification
        f.write('"BMI_Classification": [\n')
        first = True
        for item in gen_bmi_classification():
            if not first:
                f.write(",\n")
            f.write(json.dumps(item, ensure_ascii=False))
            first = False
        f.write("\n],\n")

        # 2. Waist_Risk
        f.write('"Waist_Risk": [\n')
        first = True
        for item in gen_waist_risk():
            if not first:
                f.write(",\n")
            f.write(json.dumps(item, ensure_ascii=False))
            first = False
        f.write("\n],\n")

        # 3. Diseases
        f.write('"Diseases": [\n')
        f.write(",\n".join(json.dumps(d, ensure_ascii=False) for d in diseases))
        f.write("\n],\n")

        # 4. Foods
        f.write('"Foods": [\n')
        food_ids = []
        first = True
        for food in gen_food_items():
            if not first:
                f.write(",\n")
            f.write(json.dumps(food, ensure_ascii=False))
            food_ids.append(food["id"])
            first = False
        f.write("\n],\n")

        # 5. Meal_Plans
        f.write('"Meal_Plans": [\n')
        first = True
        for mp in gen_meal_plans(food_ids):
            if not first:
                f.write(",\n")
            f.write(json.dumps(mp, ensure_ascii=False))
            first = False
        f.write("\n],\n")

```

```

# 6. Exercise
f.write('"Exercise": [\n')
first = True
for ex in gen_exercises():
    if not first:
        f.write(",\n")
    f.write(json.dumps(ex, ensure_ascii=False))
    first = False
f.write("\n],\n")

# 7. Rules (this is the heavy part)
print("Writing rules (this may take a while)...")
f.write('"Rules": [\n')
first = True
# define pools for rules generation
age_groups = [(18,30),(31,40),(41,50),(51,60),(61,80)]
genders = ["Male","Female"]
activities = ["Low","Moderate","High","Very High"]

rule_gen = gen_rules(n_bmi_steps, age_groups, genders, diseases, activities, n_rules_target)
chunk = 0
for rule in rule_gen:
    if not first:
        f.write(",\n")
    f.write(json.dumps(rule, ensure_ascii=False))
    first = False
    chunk += 1
    if chunk % 100000 == 0:
        print(f" - written {chunk} rules at {datetime.now().isoformat()}")
f.write("\n]\n") # close Rules
f.write("\n") # close root object
print("Finished KB build:", datetime.now().isoformat())
print("Saved to:", outpath)

```

```

# --- Simple KB query to produce a short health guide for a user ---
def generate_health_guide(user_profile, metrics_df=None, kb_path=OUTPATH, max_rules_to_scan=100000):
    """
    user_profile: dict with keys including 'age', 'gender', 'BMI', 'disease' (optional), 'activity_level'
    metrics_df: pandas DataFrame with model metrics (from Part F) to include in the guide
    This function streams through the Rules in the KB and returns matching advice (first matches).
    NOTE: streaming large JSON is slower; we scan Rules only to avoid loading entire KB.
    """
    matches = []
    advice_set = []
    target_bmi = user_profile.get("BMI", None)
    target_age = user_profile.get("age", None)
    target_gender = user_profile.get("gender", None)
    target_disease = user_profile.get("disease", None)
    target_activity = user_profile.get("activity_level", None)

```

```

# We'll parse the JSON file and locate "Rules" array start, then stream-read rules line by line.
found_rules = 0
with open(kb_path, "r", encoding="utf-8") as f:
    # Find the start of the "Rules" array
    for line in f:
        if '"Rules": [' in line:
            break
    # Now read subsequent lines until the end of array ']\n'
    for line in f:
        s = line.strip()
        if s.startswith(']'):
            break
        # remove trailing commas
        if s.endswith(','):
            s = s[:-1]
        if not s:
            continue
        try:
            rule = json.loads(s)
        except Exception:
            # if object spans multiple lines, try to read full object
            # accumulate until balanced braces
            obj_lines = [s]
            braces = s.count("{") - s.count("}")
            while braces > 0:
                nxt = next(f)
                obj_lines.append(nxt)
                braces += nxt.count("{") - nxt.count("}")
            obj_text = "".join(obj_lines)
            rule = json.loads(obj_text.rstrip().rstrip(","))
        found_rules += 1

```

```

cond = rule.get("if", {})
ok = True
if target_bmi is not None and "BMI" in cond:
    try:
        # cond BMI stored as string like "24.5"
        cond_bmi = float(cond["BMI"])
        # allow +/- 0.5 tolerance
        if abs(cond_bmi - target_bmi) > 1.0:
            ok = False
    except:
        pass
if ok and target_age is not None and "age" in cond:
    lo,hi = map(int, cond["age"].split("-"))
    if not (lo <= target_age <= hi):
        ok = False
if ok and target_gender is not None and "gender" in cond:
    if cond["gender"] != target_gender:
        ok = False
if ok and target_disease is not None and "disease" in cond:
    # only check if disease present and matches
    if cond.get("disease") and cond["disease"] != target_disease:
        ok = False
if ok:
    matches.append(rule)
    for t in rule.get("then", {}).get("advice", []):
        advice_set.append(t)
if found_rules >= max_rules_to_scan:
    break

# Compose the guide
guide = {
    "user_profile": user_profile,
    "summary": f"Found {len(matches)} matching rules (scanned {found_rules}).",
    "top_advice": advice_set[:10]
}

# Integrate metrics from Part F
if metrics_df is not None:
    # metrics_df expected to have Model, Accuracy columns
    best_row = metrics_df.sort_values(by="Accuracy", ascending=False).iloc[0]
    guide["model_recommendation"] = {
        "best_model": best_row["Model"],
        "accuracy": float(best_row["Accuracy"])
    }

return guide

```

```

# -----
# Example usage (do not run if you only want to generate KB)
# -----
if __name__ == "__main__":
    # 1) Build KB (this may take long; tune sizes above to fit resources)
    build_kb_json(OUTPATH)

    # 2) Example generate guide (after KB built)
    # Suppose you have metrics DataFrame from Part F saved as CSV:
    # metrics_df = pd.read_csv("model_metrics_partf.csv")
    # For demo, create small metrics df:
    import pandas as pd
    metrics_df = pd.DataFrame([
        {"Model": "LSTM", "Accuracy": 0.90},
        {"Model": "CNN", "Accuracy": 0.88},
        {"Model": "Random Forest", "Accuracy": 0.85}
    ])
    user = {"age": 45, "gender": "Male", "BMI": 29.4, "disease": "Type 2 Diabetes", "activity_level": "Low"}
    print("Generating guide (this will scan a subset of rules)...")
    guide = generate_health_guide(user, metrics_df=metrics_df, kb_path=OUTPATH, max_rules_to_scan=500000)
    print(json.dumps(guide, indent=2, ensure_ascii=False))

```

```

Writing rules (this may take a while)...
- written 100000 rules at 2025-09-14T00:24:44.524682
- written 200000 rules at 2025-09-14T00:24:45.441258
- written 300000 rules at 2025-09-14T00:24:46.838050
- written 400000 rules at 2025-09-14T00:24:48.260868
- written 500000 rules at 2025-09-14T00:24:49.703349
- written 600000 rules at 2025-09-14T00:24:51.144719
- written 700000 rules at 2025-09-14T00:24:52.508065
- written 800000 rules at 2025-09-14T00:24:53.973592
- written 900000 rules at 2025-09-14T00:24:55.419608
- written 1000000 rules at 2025-09-14T00:24:56.919462
- written 1100000 rules at 2025-09-14T00:24:58.382764
- written 1200000 rules at 2025-09-14T00:24:59.773215
- written 1300000 rules at 2025-09-14T00:25:01.151903
- written 1400000 rules at 2025-09-14T00:25:02.594285
- written 1500000 rules at 2025-09-14T00:25:04.055229
- written 1600000 rules at 2025-09-14T00:25:05.447672
- written 1700000 rules at 2025-09-14T00:25:06.868583
- written 1800000 rules at 2025-09-14T00:25:08.262958
- written 1900000 rules at 2025-09-14T00:25:09.772801
- written 2000000 rules at 2025-09-14T00:25:11.324184
- written 2100000 rules at 2025-09-14T00:25:12.782391
- written 2200000 rules at 2025-09-14T00:25:14.262171
- written 2300000 rules at 2025-09-14T00:25:15.763795
- written 2400000 rules at 2025-09-14T00:25:17.281651
- written 2500000 rules at 2025-09-14T00:25:18.881132
- written 2600000 rules at 2025-09-14T00:25:20.285770
- written 2700000 rules at 2025-09-14T00:25:21.721224
- written 2800000 rules at 2025-09-14T00:25:23.122627
Finished KB build: 2025-09-14T00:25:24.273279
Saved to: D:\lecture\PTIT\20251\IntSys\kb_healthGuide.json
Generating guide (this will scan a subset of rules)...
{
  "user_profile": {
    "age": 45,
    "gender": "Male",
    "BMI": 29.4,
    "disease": "Type 2 Diabetes",
    "activity_level": "Low"
  },
  "summary": "Found 3082 matching rules (scanned 500000).",
  "top_advice": [
    "Reduce portion size, increase activity",
    "Include resistance training 2x/week",
    "Reduce portion size, increase activity",
    "Include resistance training 2x/week",
    "Reduce portion size, increase activity",
    "Reduce portion size, increase activity",
    "Reduce portion size, increase activity",
    "High risk: recommend clinical weight loss program",
    "Reduce refined carbs and added sugars",
    "Include resistance training 2x/week"
  ],
  "model_recommendation": {
    "best_model": "LSTM",
    "accuracy": 0.9
  }
}

```

1. Generates modular KB sections:

- BMI classification with fine granularity (0.1 steps).
- Waist risk by age/gender.
- A short disease list with diet/exercise suggestions.
- A synthetic food database of `n_food_items` entries.
- `n_mealplans` meal plans combining food items.
- `n_exercises` exercise entries.
- **Rules:** millions of small rule objects combining BMI, age, gender, disease, and activity to create condition→advice rules. This section is the largest to reach >3M entries.

2. **Writes JSON incrementally** into a single JSON file with top-level keys mapping to arrays, so the final file is a valid JSON object but produced without holding everything in memory.

3. **Provides generate_health_guide:** a streaming rule-scanner that reads the Rules array from the big JSON file and returns a short personalized guide based on the user's profile and the Part F metrics (which you pass as metrics_df).